

# Protocol Layering

Network software is organised into layers so that each layer can be designed, replaced and understood independently. In the Internet protocol stack the layers are the application layer, the transport layer, the network layer, the link layer and the physical layer. Each layer offers a service to the layer above it and uses the service of the layer below it.

When an application sends data, every layer adds its own header in a process called encapsulation. The transport layer adds a header with port numbers to form a segment. The network layer adds an IP header with source and destination addresses to form a packet. The link layer adds a frame header and trailer so that the packet can cross one link. The receiver removes the headers in the opposite order, which is called decapsulation.

The transport layer provides communication between processes on different hosts. TCP offers a reliable, ordered byte stream, while UDP offers simple, unreliable datagrams. The network layer provides delivery of packets between hosts and is responsible for forwarding and routing. Routers operate up to the network layer, whereas link layer switches forward frames inside a single network.

# TCP Connections and Reliability

TCP is connection oriented. A connection is established with a three-way handshake. The client sends a SYN segment carrying its initial sequence number. The server replies with a SYN-ACK segment that acknowledges the client's number and carries its own initial sequence number. The client completes the handshake with an ACK segment. After this exchange both sides are ready to send data.

TCP numbers every byte of the stream with a sequence number. The acknowledgment number tells the sender the next byte the receiver expects. Acknowledgments are cumulative, so an acknowledgment for byte 5000 confirms every byte before 5000.

The sender starts a retransmission timer for unacknowledged data. If the timer expires before an acknowledgment arrives, the sender retransmits the segment. The timeout value is derived from measurements of the round-trip time, so it adapts to the path. A connection is closed with FIN segments, and each direction of the connection is closed separately.

# TCP Flow Control

Flow control prevents a fast sender from overflowing the receive buffer of a slow receiver. It protects the receiver, not the network. The receiver advertises how much free buffer space it has in the receive window field of every segment it sends, often abbreviated `rwnd`.

The sender keeps the amount of unacknowledged data no larger than the advertised receive window. As the application reads data from the buffer, the receiver advertises a larger window in later acknowledgments. If the receiver's buffer becomes full it advertises a window of zero and the sender pauses. The sender then sends small probe segments so that it learns when the window opens again.

Flow control and congestion control are different mechanisms. Flow control is about the receiver's buffer, while congestion control is about the capacity of the network between the hosts. A TCP sender obeys both limits at the same time.

# TCP Congestion Control

TCP congestion control limits how fast a sender puts data into the network so that routers are not overwhelmed. The sender maintains a variable called the congestion window, written `cwnd`, which is the amount of unacknowledged data the sender may have in flight. The sender never has more than the smaller of `cwnd` and the receiver's advertised window outstanding.

A connection begins in slow start with a small congestion window. During slow start the sender increases `cwnd` by one maximum segment size for every acknowledgment received, which roughly doubles `cwnd` every round-trip time. The growth is exponential despite the name. Slow start continues until `cwnd` reaches the slow start threshold, called `ssthresh`, or until a loss is detected.

Once `cwnd` reaches `ssthresh`, TCP switches to congestion avoidance. In congestion avoidance `cwnd` grows by about one maximum segment size per round-trip time, so the growth is linear rather than exponential. This is why the window stops doubling at the threshold.

Most TCP variants treat packet loss as a sign of congestion. There are two loss signals. A timeout is the more serious signal, and three duplicate acknowledgments is the milder one. In TCP Reno, three duplicate acknowledgments trigger fast retransmit, in which the sender retransmits the missing segment without waiting for the timer. Fast recovery then sets `ssthresh` to half of the amount of data in flight and continues in congestion avoidance instead of restarting from the beginning.

After a timeout, TCP sets `ssthresh` to half of the current `cwnd`, sets `cwnd` back to one maximum segment size, and returns to slow start. The combination of additive increase and multiplicative decrease is known as AIMD, and it lets competing flows converge toward a fair share of a bottleneck link. Packet loss is not always caused by congestion, for example on wireless links, which is why loss based congestion control can underuse such paths.

# IP Addressing and Subnetting

An IPv4 address has 32 bits and is written as four decimal numbers separated by dots. An address has a network prefix and a host part. CIDR notation writes the prefix length after a slash, so 192.168.10.0/24 means that the first 24 bits identify the network and the last 8 bits identify hosts.

A subnet mask has ones in the prefix positions and zeros in the host positions. The mask for a /24 prefix is 255.255.255.0. A network with  $h$  host bits contains  $2^h$  addresses. Two of them are reserved, the all zeros host part for the network address and the all ones host part for the broadcast address.

A /26 prefix leaves 6 host bits, so each subnet has 64 addresses and 62 usable host addresses. Splitting a /24 network into /26 subnets therefore produces four subnets. A router uses the longest matching prefix in its forwarding table to choose the next hop.

# Routing

Routing is the process of determining the path that packets take from a source to a destination, while forwarding is the action of moving a packet to the next hop. A router builds a forwarding table from the information produced by routing protocols.

In distance vector routing each router tells its neighbours the cost of its best known path to every destination. Routers update their tables using the Bellman-Ford equation. Distance vector protocols can suffer from the count to infinity problem when links fail.

In link state routing each router floods information about the state of its links to every other router. Every router then has the same view of the topology and computes shortest paths with Dijkstra's algorithm. OSPF is a widely used link state protocol inside a single administrative domain. Between administrative domains the Internet uses BGP, a path vector protocol in which policy matters as much as shortest paths.